Strategy
Observer

T120B516 Objektinis programų projektavimas

2026 rudens semestras

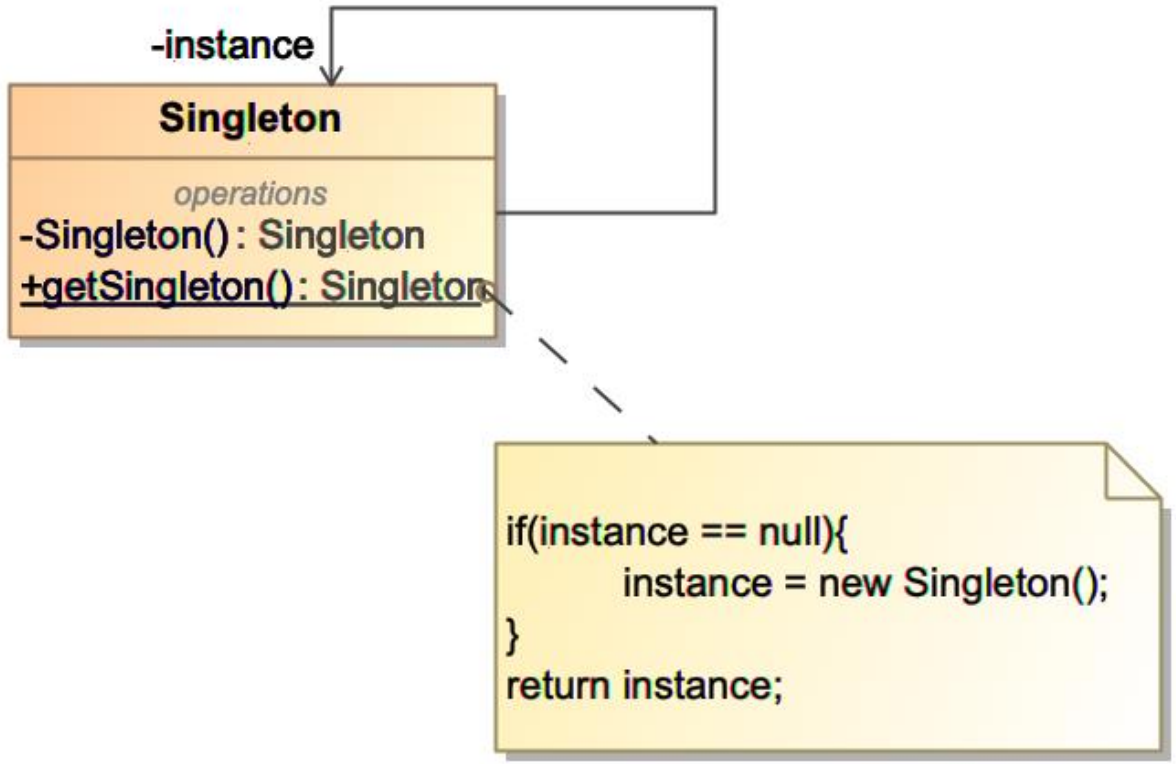
Dr. Andrej Ušaniov

Previous lecture

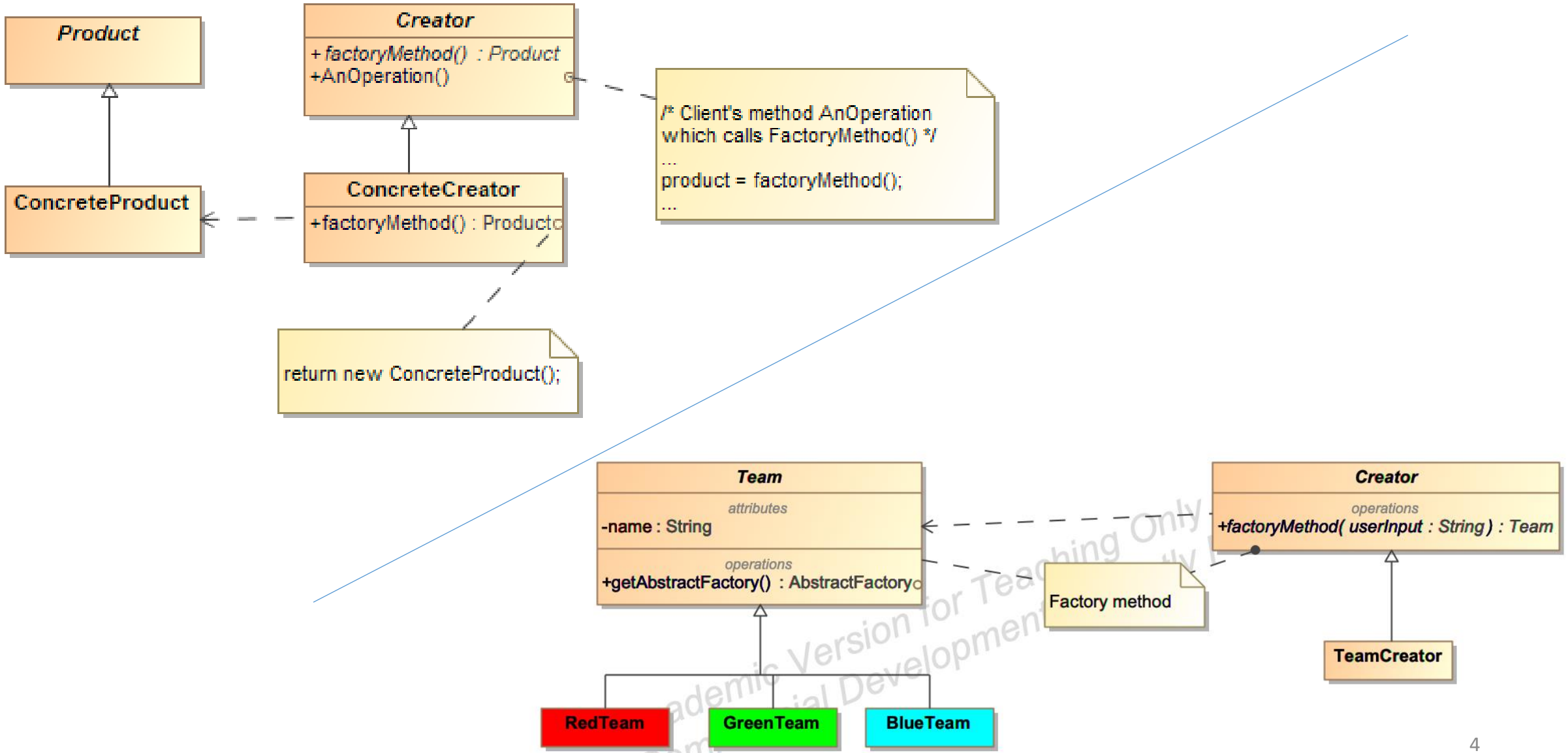
- Singleton
- Factory Method
- Abstract Factory

Previous lecture: Singleton

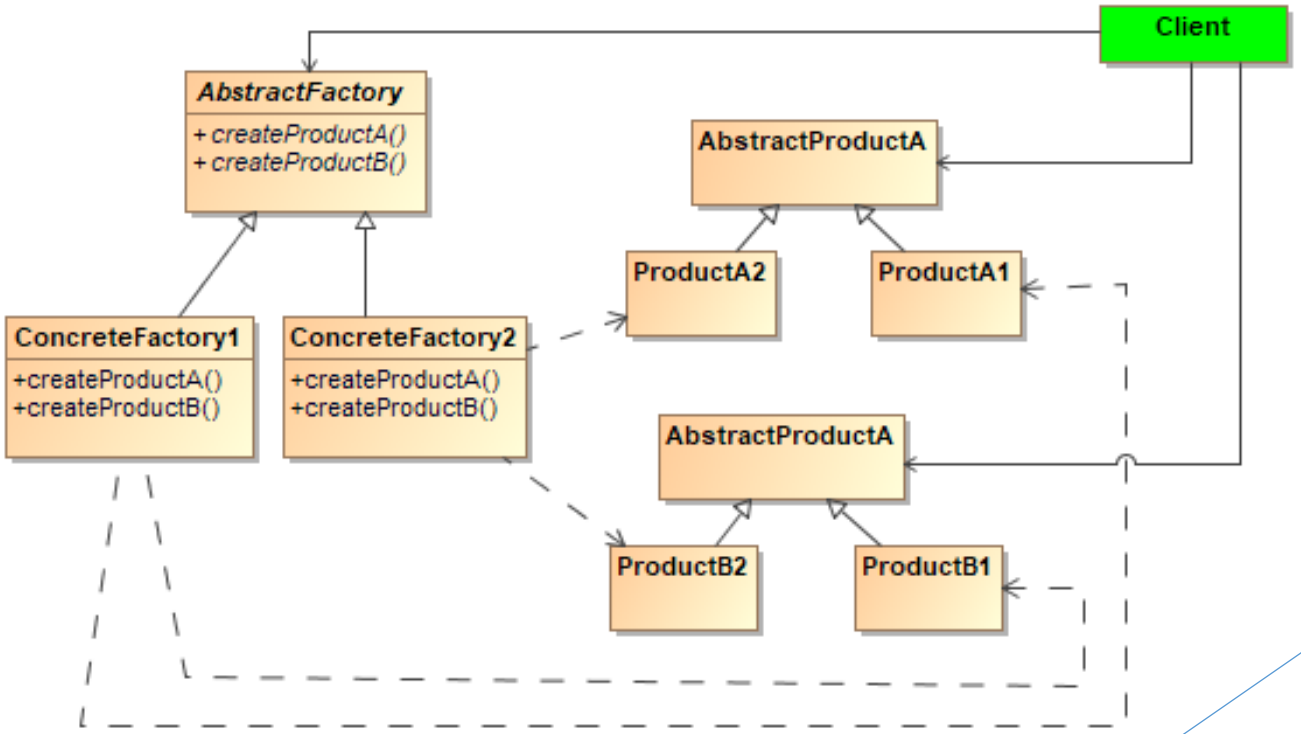
- Returns unique instance

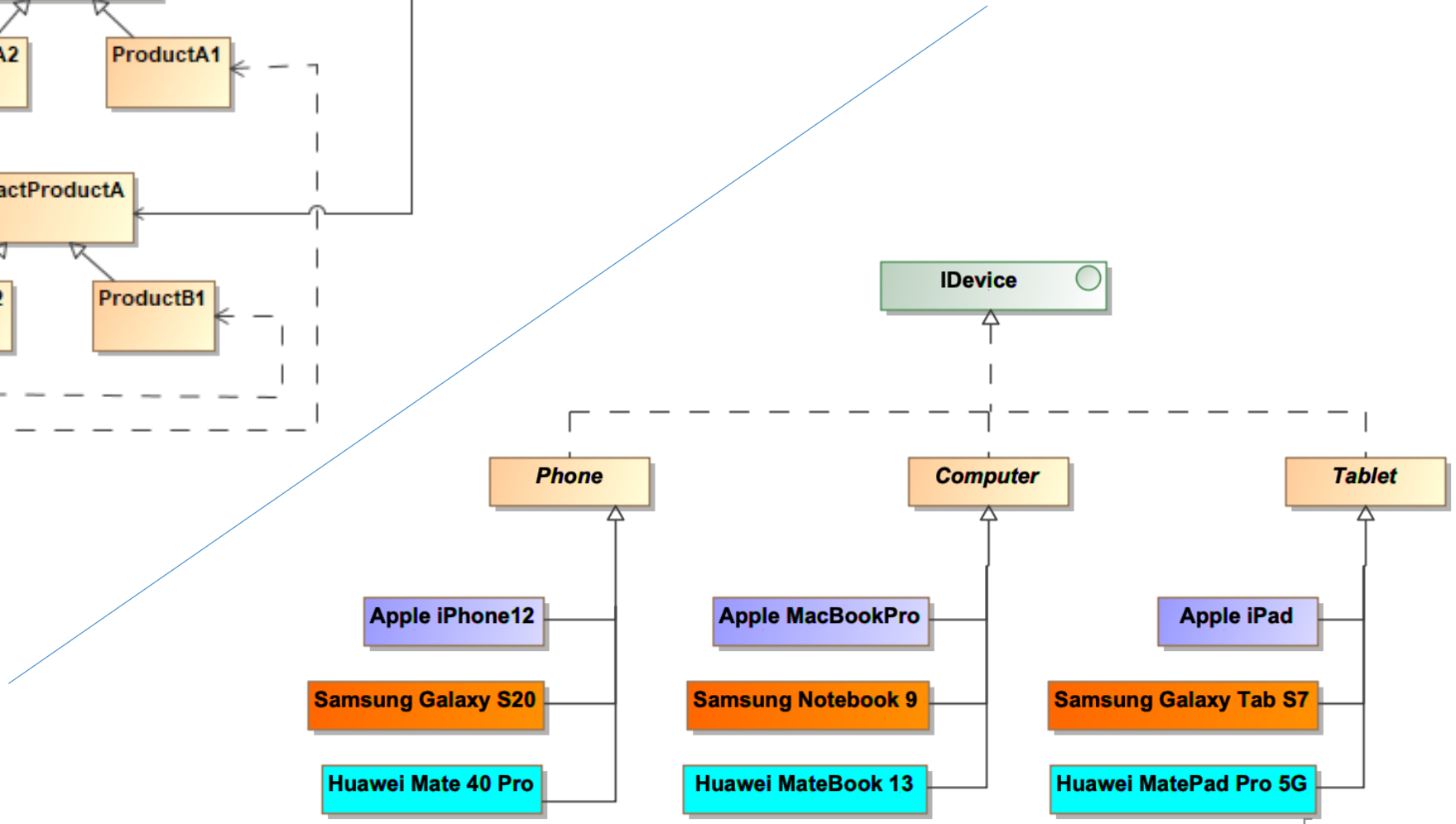


Previous lecture: Factory Method



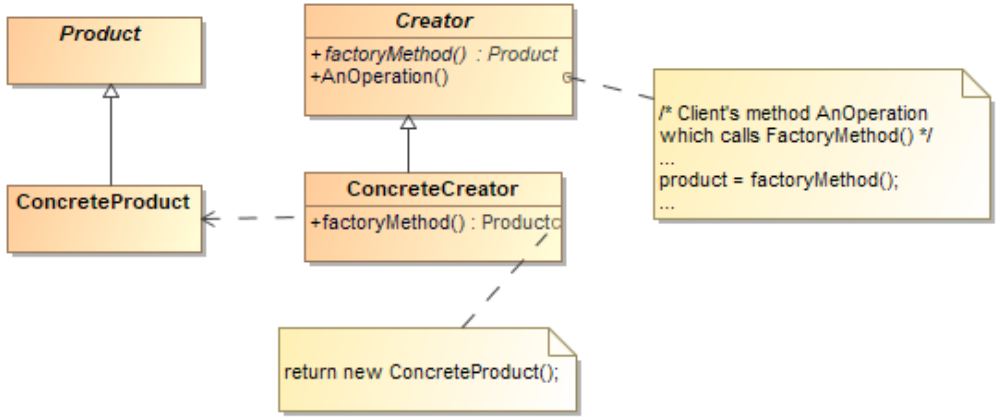
Previous lecture: Abstract Factory

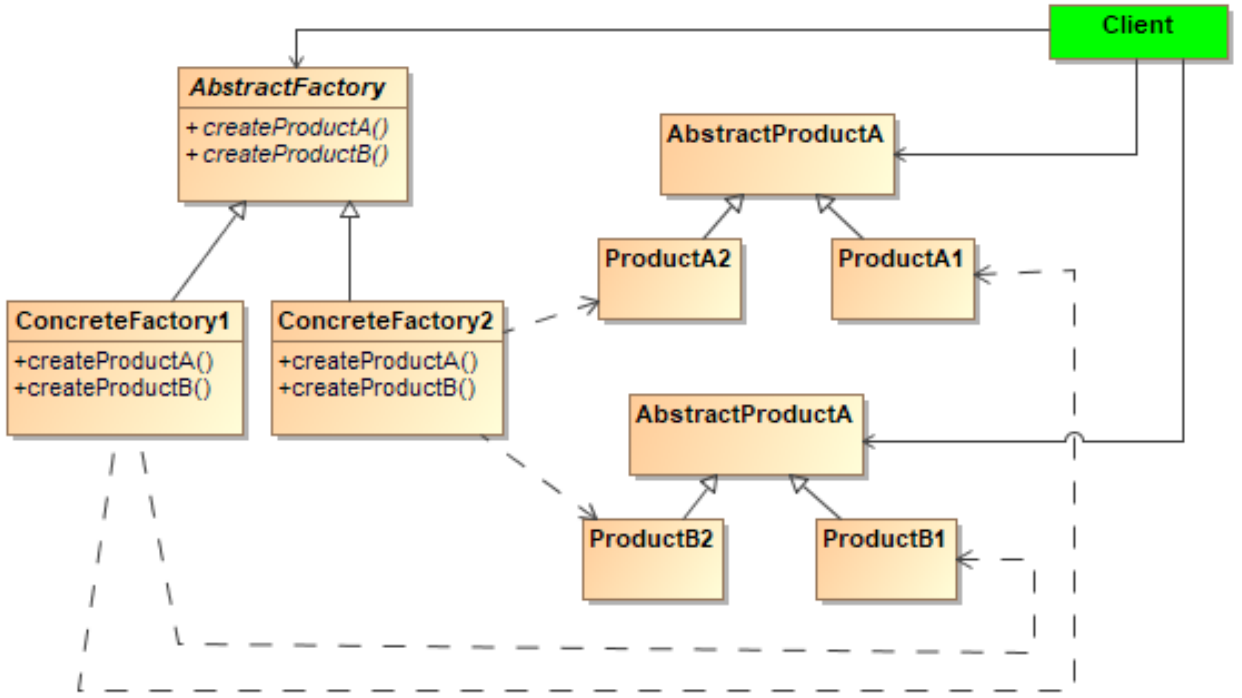




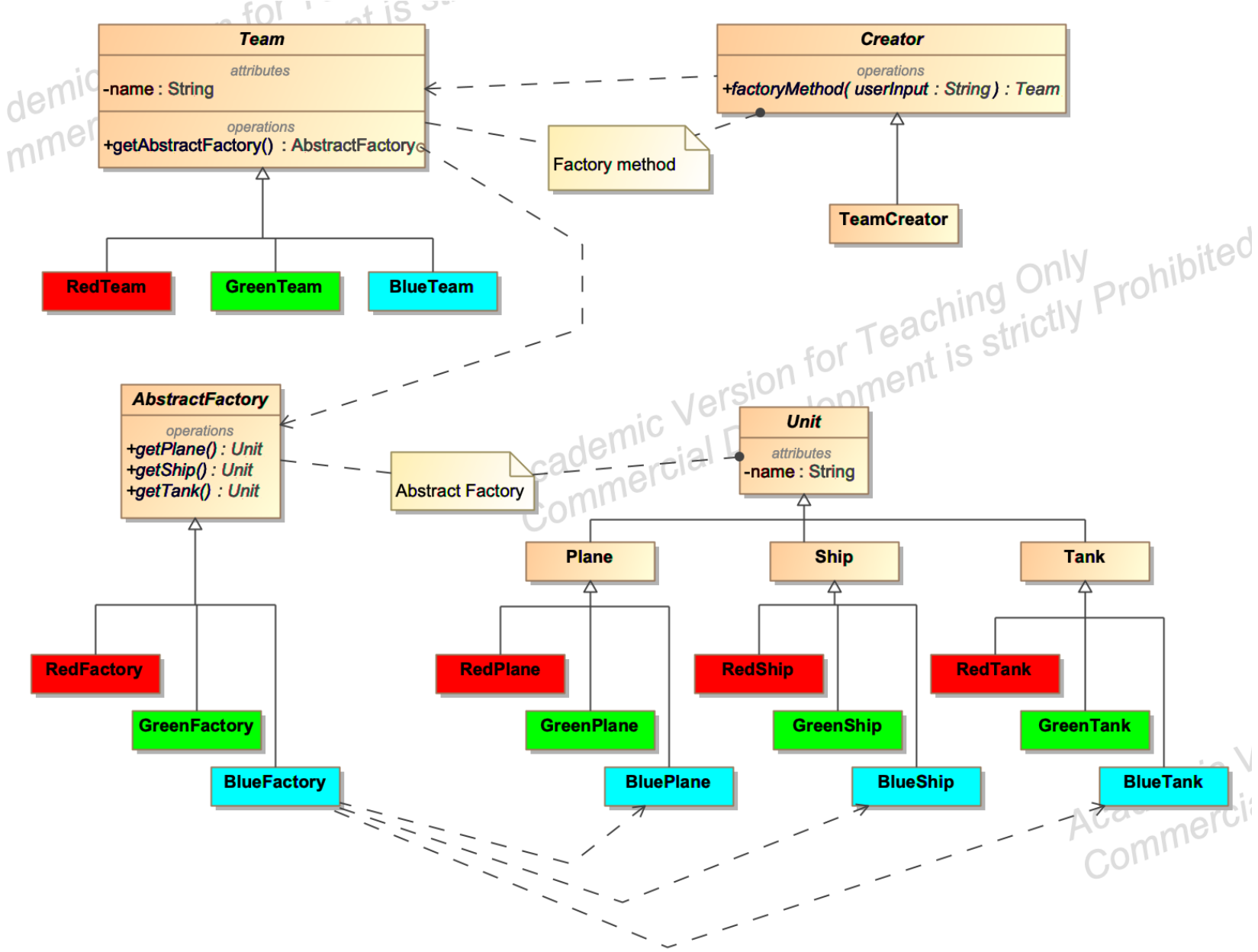
Factory method vs. Abstract Factory

- The Factory Method is just a method, which can be overridden in a subclass
- The Abstract Factory is an object that has multiple factory methods on it





Previous lecture result



Player moka bėgti, plaukti ir skristi. Kiek `if` mums reikės?

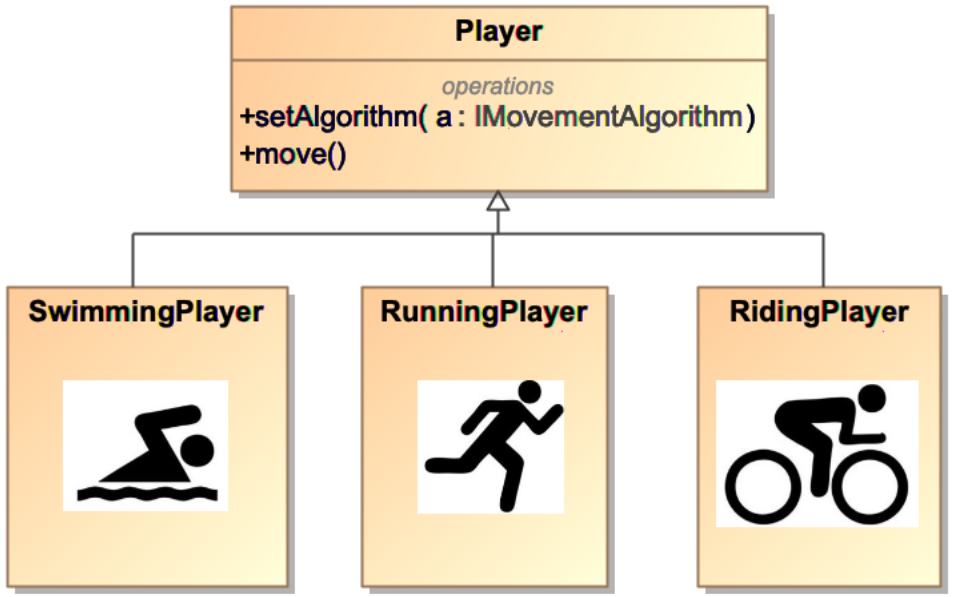
```
move() {
   if (...) run();
   else if (...) swim();
   else if (...) fly();
   else if (...) jump();
   ...
}
```

O jeigu rytoj bus 15 elgsenų?

Strategy

- Type: Behavioral
- Also Known As: Policy
- Intent: “Define **a family of algorithms**, encapsulate each one, and make them interchangeable. Strategy lets the algorithm vary independently from clients that use it.”

Suvalgė obuolį ir išmoko plaukti. Ar kursime SwimmingPlayer?



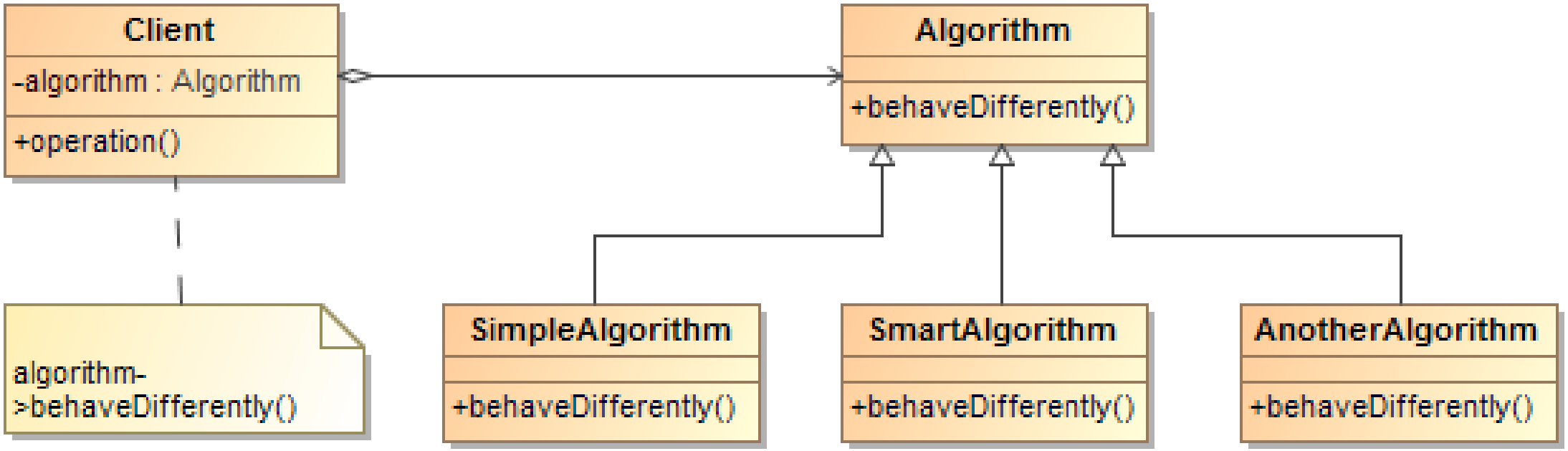
Kokius ižvelgiat privalumus ir trūkumus šiame sprendime?

O jeigu rytoj išmoksta bėgti, ką darom?
SwimmingRunningPlayer?

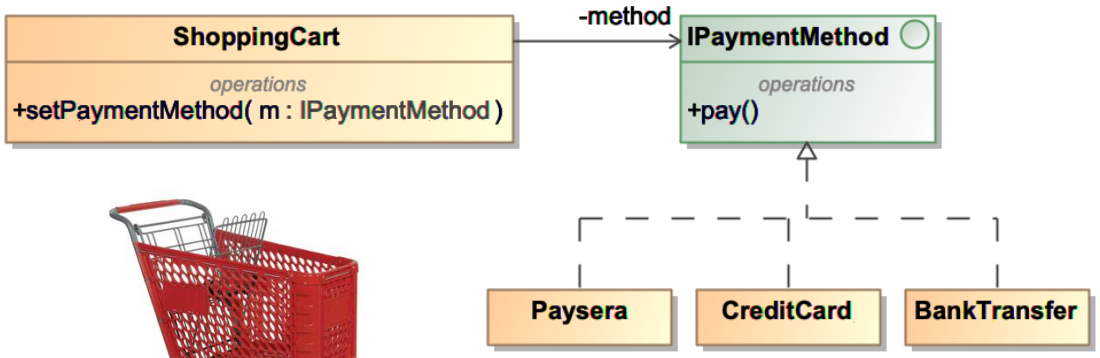
Inheritance explosion

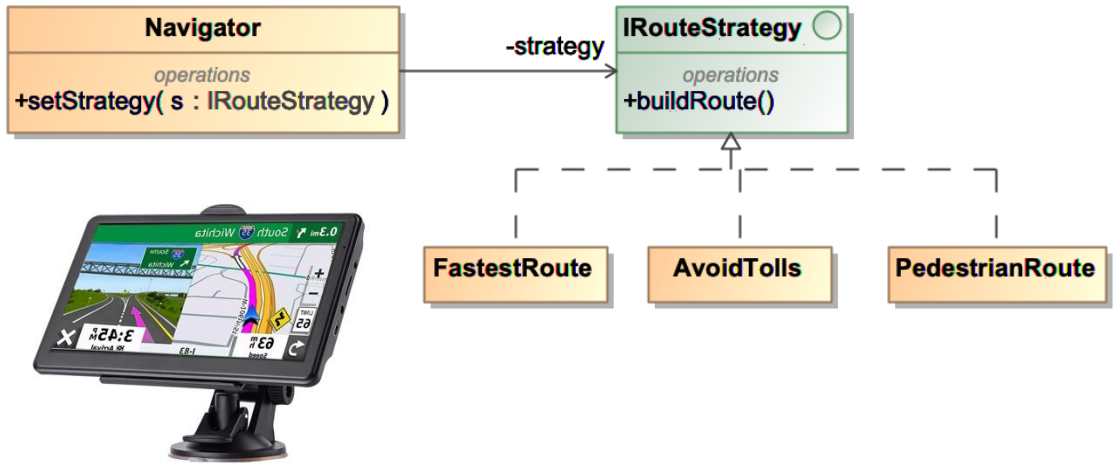
Kaip „obuolio“ problemą sprendžia GoF

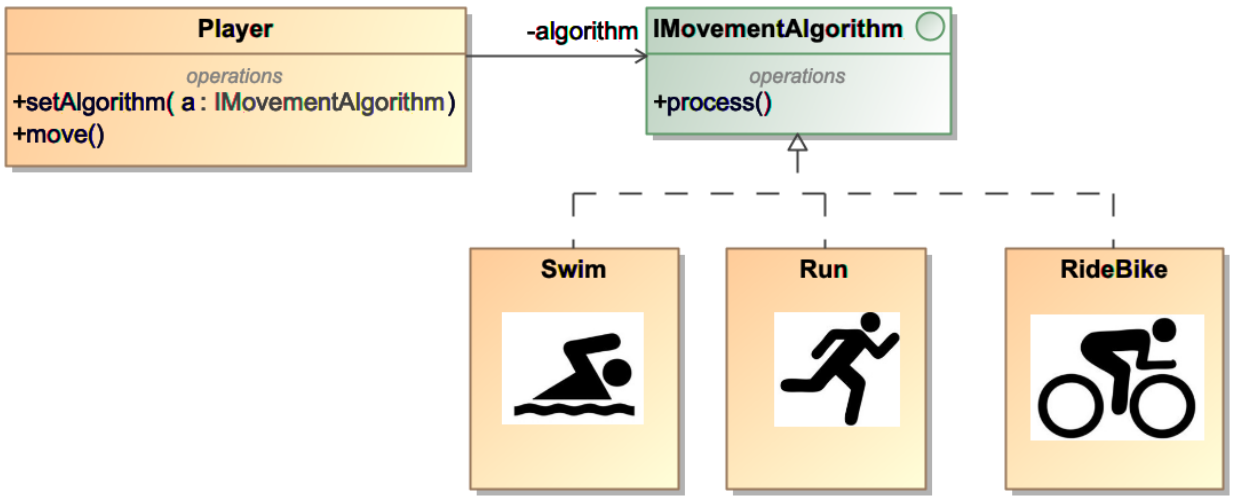
- Strategy’s structure (GoF)



Strategy real world examples



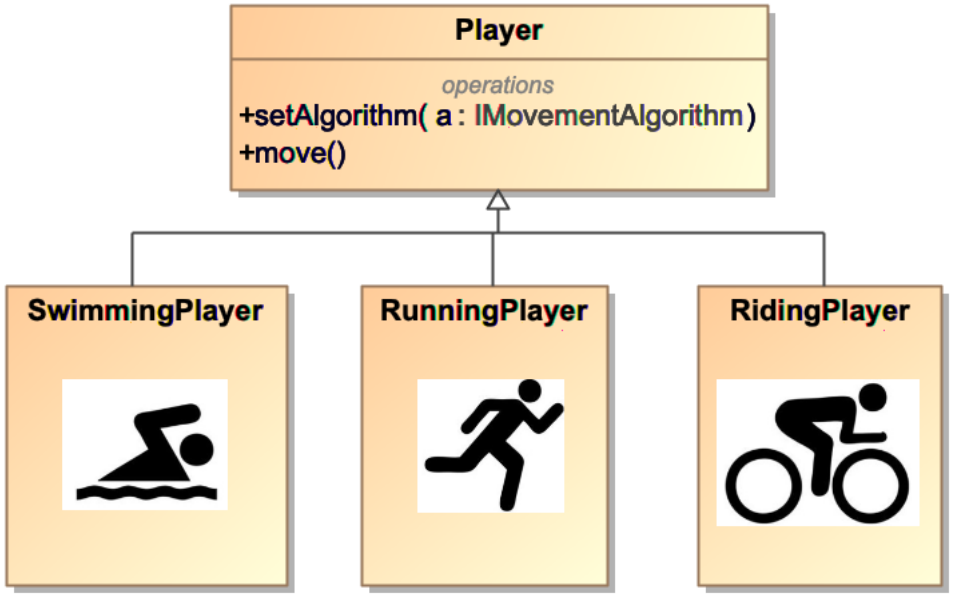




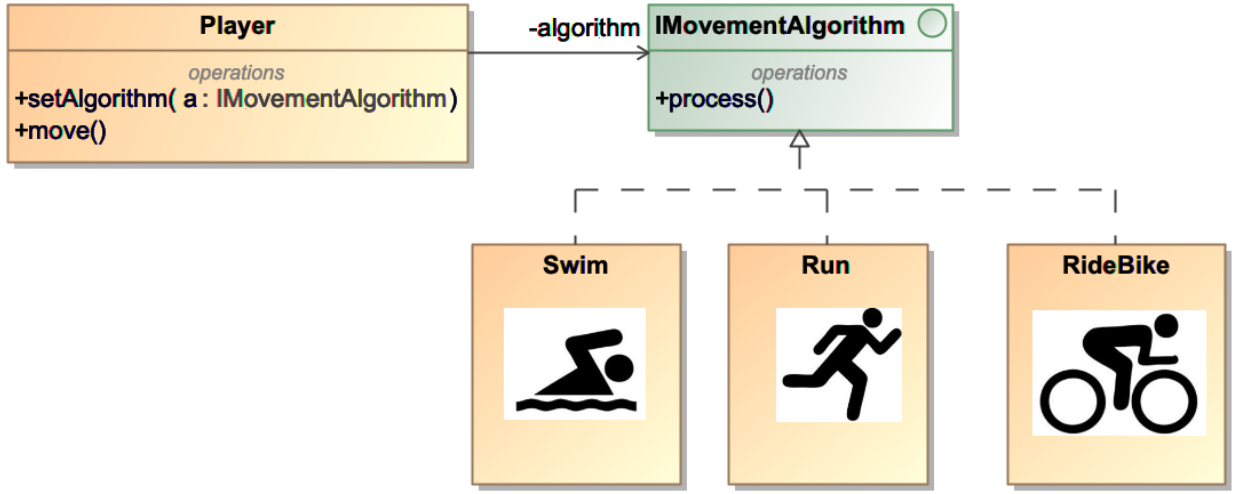
Per 10 sekundžių pasakykite, kuri diagrama neturi trūkumų

Kurį variantą pasirinktumėt dabar?

Is-a



Has-a



Taigi, kada mums praverčia Strategy?

- objektui **reikia kelių elgsenos variantų**;
- elgsena pradeda virsti į **ilgą if-else / switch-case grandinę**;
- norime elgseną **pakeisti programos vykdymo metu (runtime)**;
- norime **algoritmą atskirti nuo objekto,** kuris jį naudoja.

- Mūsų pavyzdyje: run(), swim(), fly() yra ne skirtingi Player tipai, o keičiamos judėjimo strategijos (has-a pranašumas).

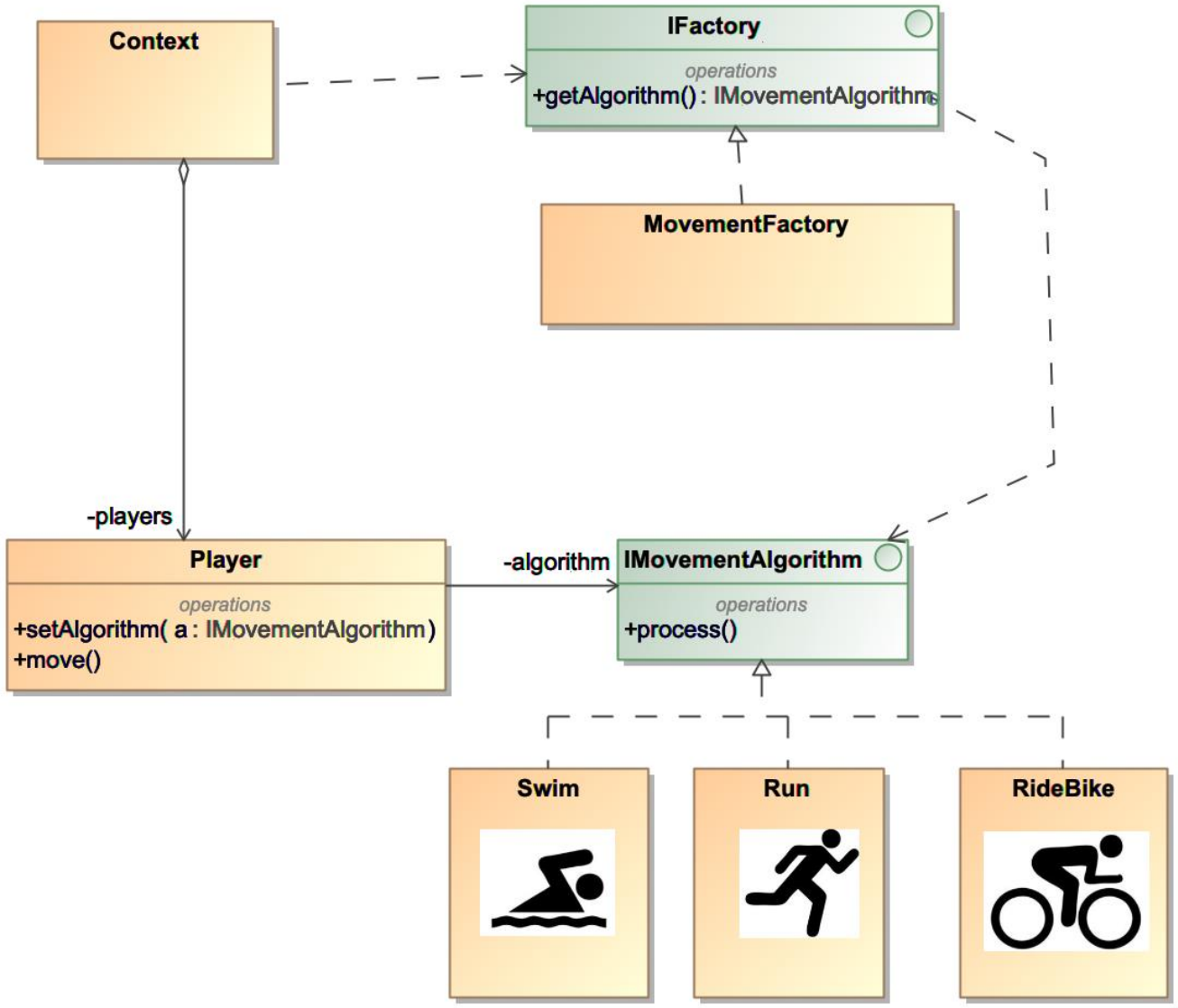
Player įšoko į vandenį. Kas turi pakeisti jo Strategy?

kas apskritai nusprendžia,

kada strategiją pakeisti?

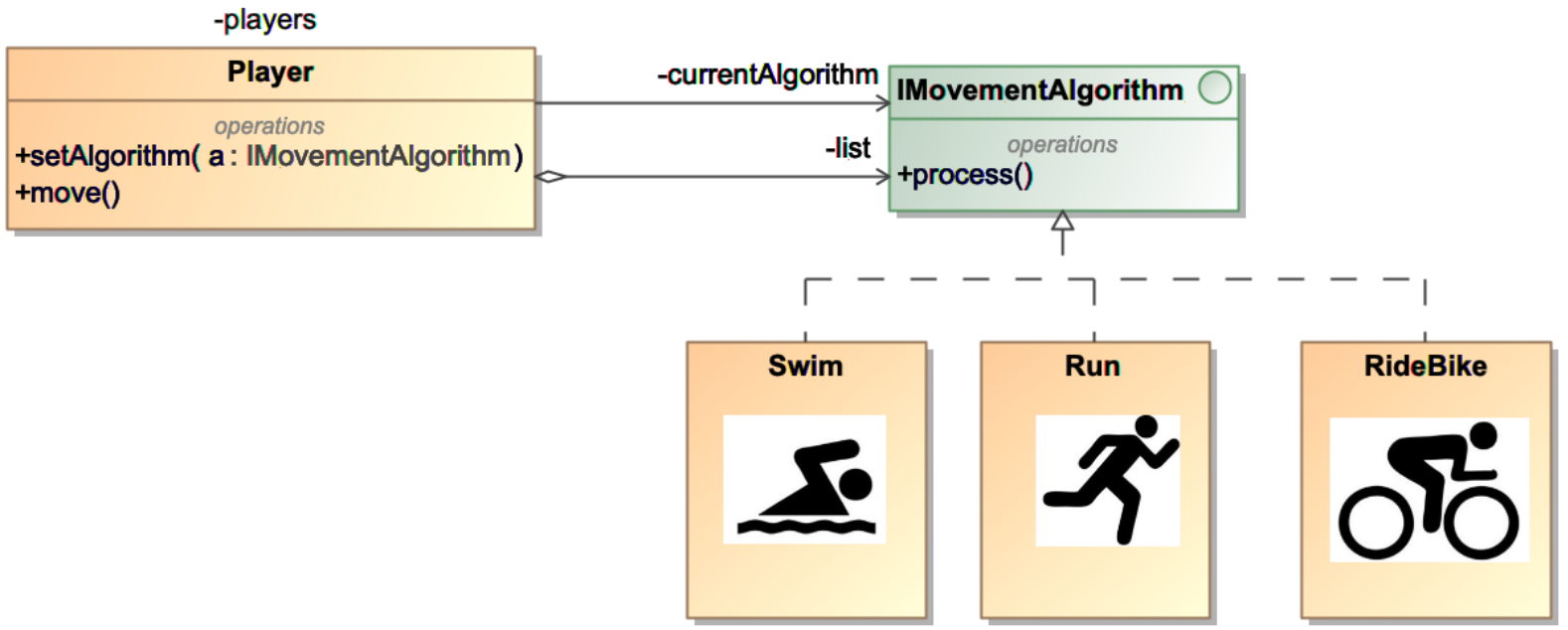
Žaidėjas? Aplinka/ Klientas?

Algoritmas? Factory?



Ar žaidėjas gali „įgyti“ kelias elgsenas žaidimo metu?

- Ar aktyvi gali būti viena strategija, o žinomų strategijų daug?



Kodėl visur nenaudojame Strategy?

✅ Elgseną galime keisti **runtime**.

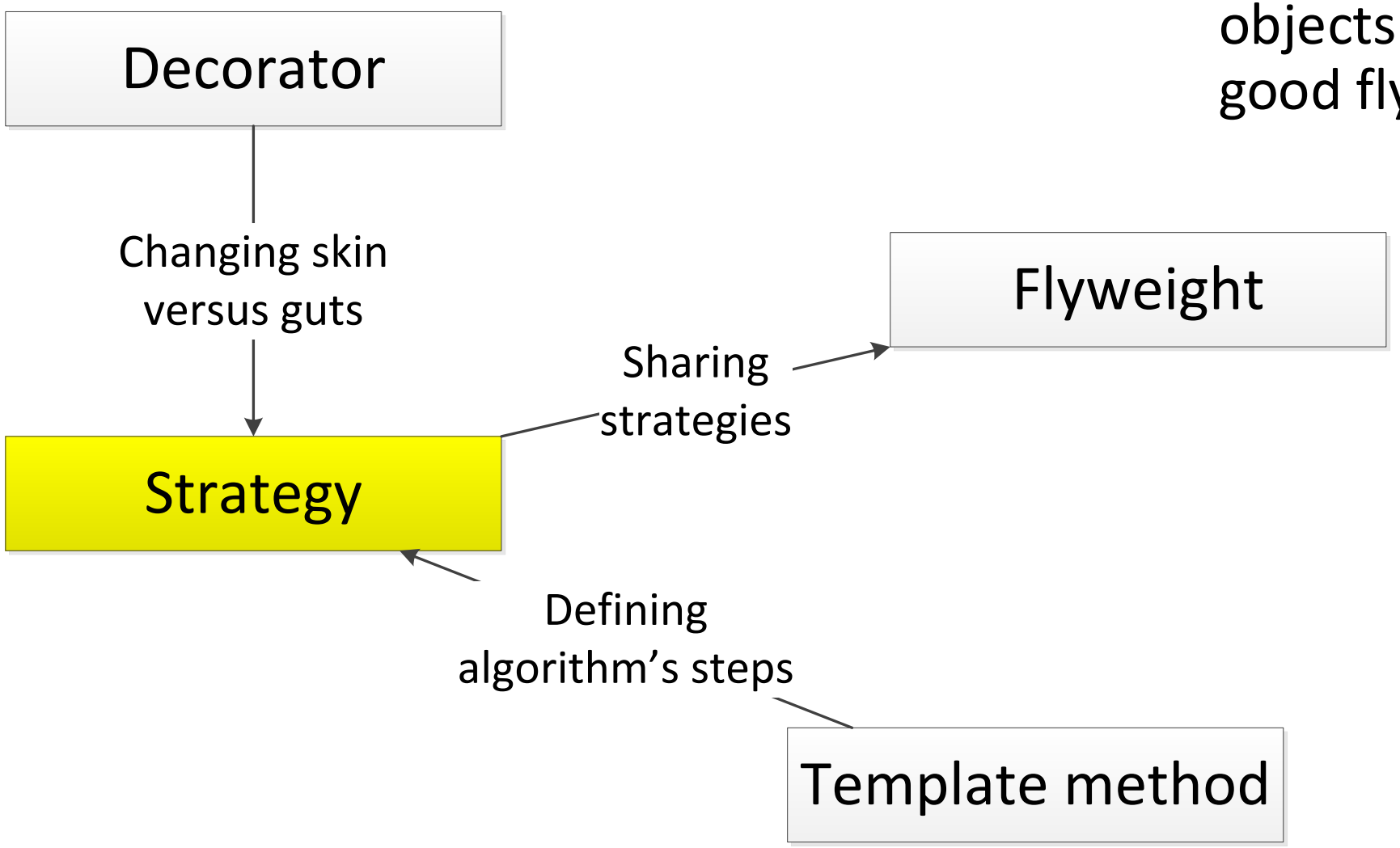
✅ Algoritmo pokytis ar naujos strategijos atsiradimas nekeičia žaidėjo.

❌ Daugėja klasių ir dėl dviejų algoritmų gal neverta taikyti šabloną.

❌ Kažkas turi žinoti skirtumus tarp strategijų ir pasirinkti tinkamą.

Strategy: related patterns

- Flyweight: Strategy objects often make good flyweights



Bonus

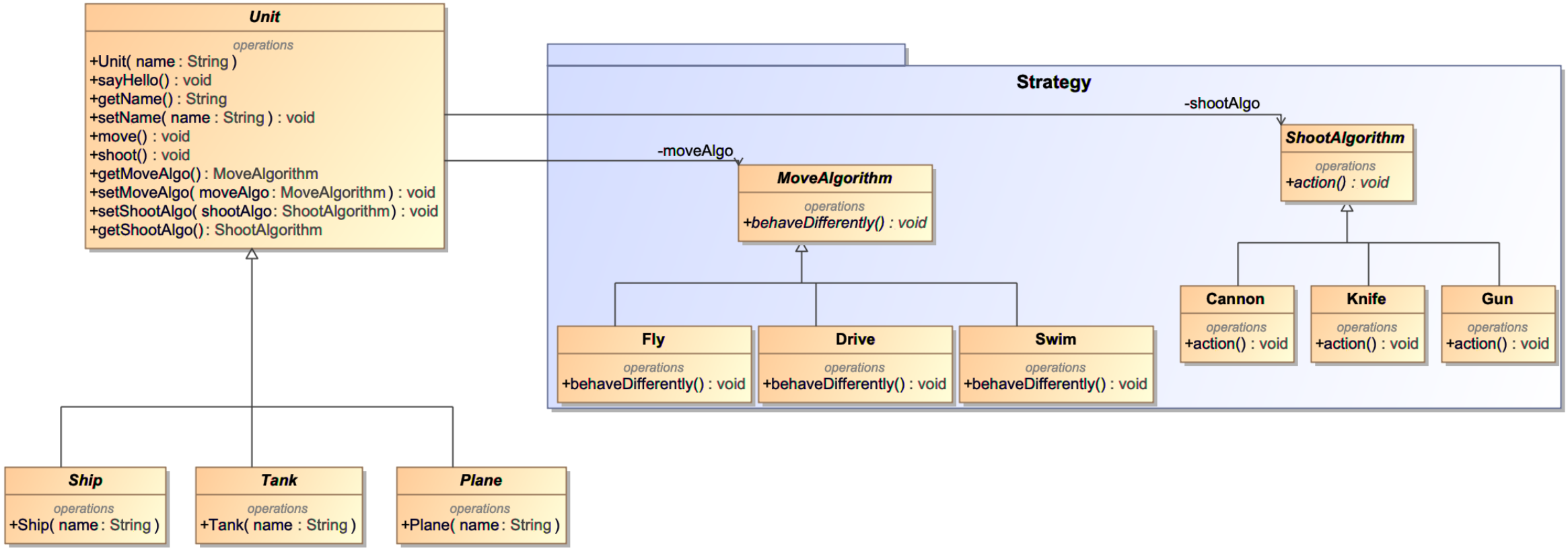
Strategijos šablono taikymas Java **Functional Interface** ir **Lambda** išraiškas

```
public class Client {
    interface Algorithm {
        int behaveDifferently(int a, int b);
        default Algorithm swap() {
          return (a, b) -> behaveDifferently(b, a); }
    }
    public static int operation(int a, int b, Algorithm op) {
        return op. behaveDifferently(a, b);
    }
    public static void main(String args) {
        Algorithm simpleAlgorithm = (a, b) -> a + b;
        Algorithm smartAlgorithm = (a, b) -> a - b;
        Algorithm anotherAlgorithm = (a, b) -> a * b;
        System.out.println("40 + 2 = " + operation(40, 2, simpleAlgorithm));
        System.out.println("20 - 10 = " + operation(20, 10, smartAlgorithm));
        System.out.println("10 - 20 = " + operation(20, 10,
anotherAlgorithm.swap()));
    }
}
```

Strategy: Example to code

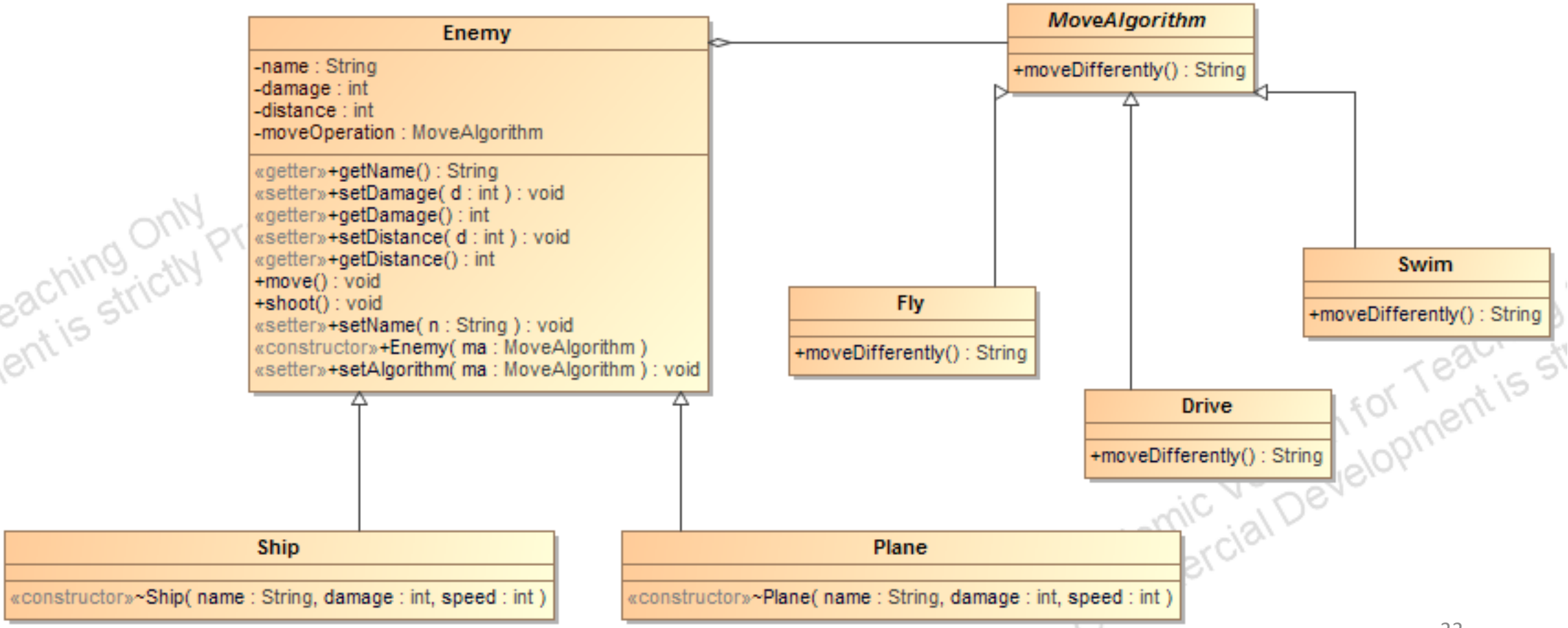
- Add different movement algorithms to our units:
  - drive, fly, swim, move, fire

Strategy example created during lecture

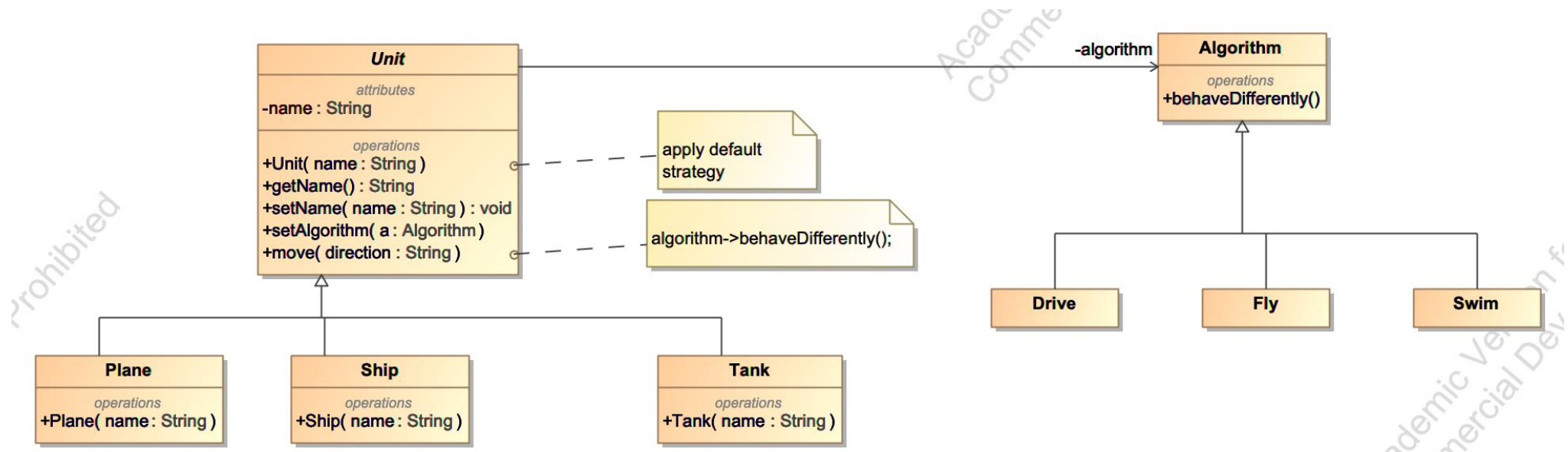


Strategy: example to code

- Add behavior to Enemy
- **Change moving algorithm: add fly, swim, drive*



Created during lecture: Strategy



Kas bendro tarp Teams grupės, orų prognozės programėles ir žaidimo serverio?

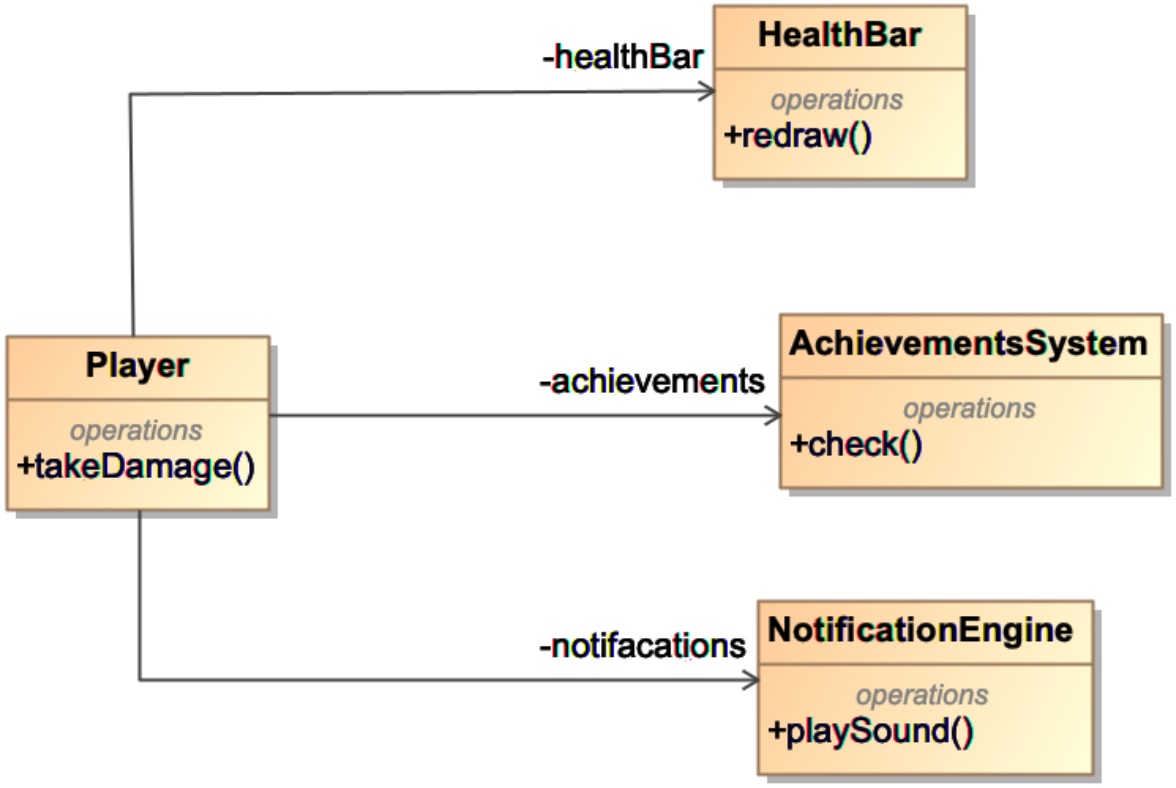
- Teams grupė → Nariai
- Orų stotis → Programėlės telefone
- Game Server → Žaidėjai

Apie pokyčius reikia informuoti tuos, kuriems tai aktualu

Observer =
„Pasikeičiau. Pranešu tiems, kam tai rūpi.“

- Type: Behavioral
- Also Known As: Dependents, Publish-Subscribe
- Intent: “Define a **one-to-many dependency** between objects so that when one object changes state, all its dependents are notified and updated automatically.”

Kai žaidėjas gauna žalos arba pasigydo, apie tai turi sužinoti keli žaidimo elementai



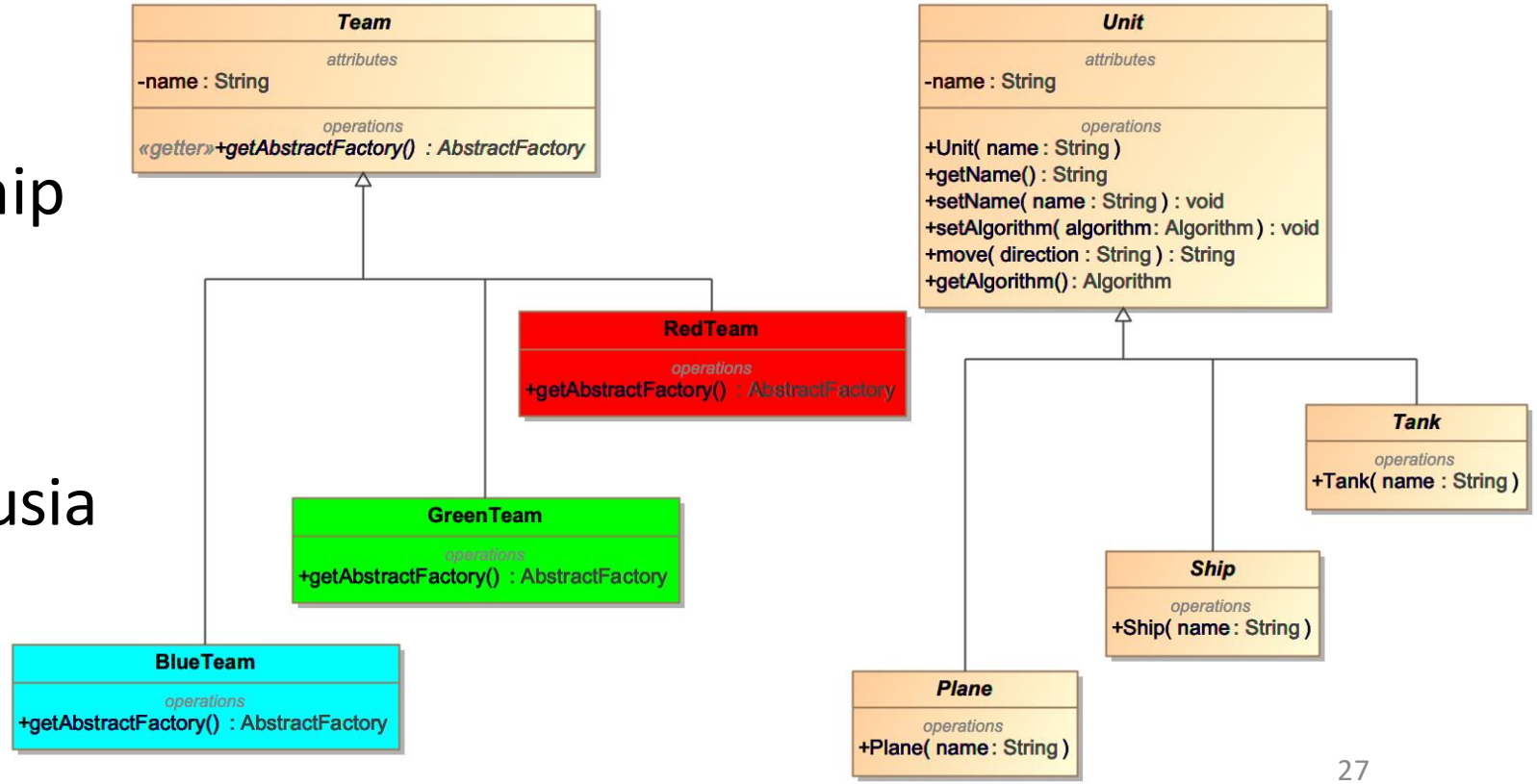
- Kas čia blogai?
- Kodas perkrautas priklausomybėmis
- O jei reikia elementą išjungti arba įjungti?
- O jei atsiras dar keli elementai?
- Dependency explosion

Komanda nori valdyti savo unit'us. Kaip unit'ai sužinos ką reikia daryti?

- Ar Team turi tiesiogiai žinoti apie kiekvieną Tank, Plane, Ship?

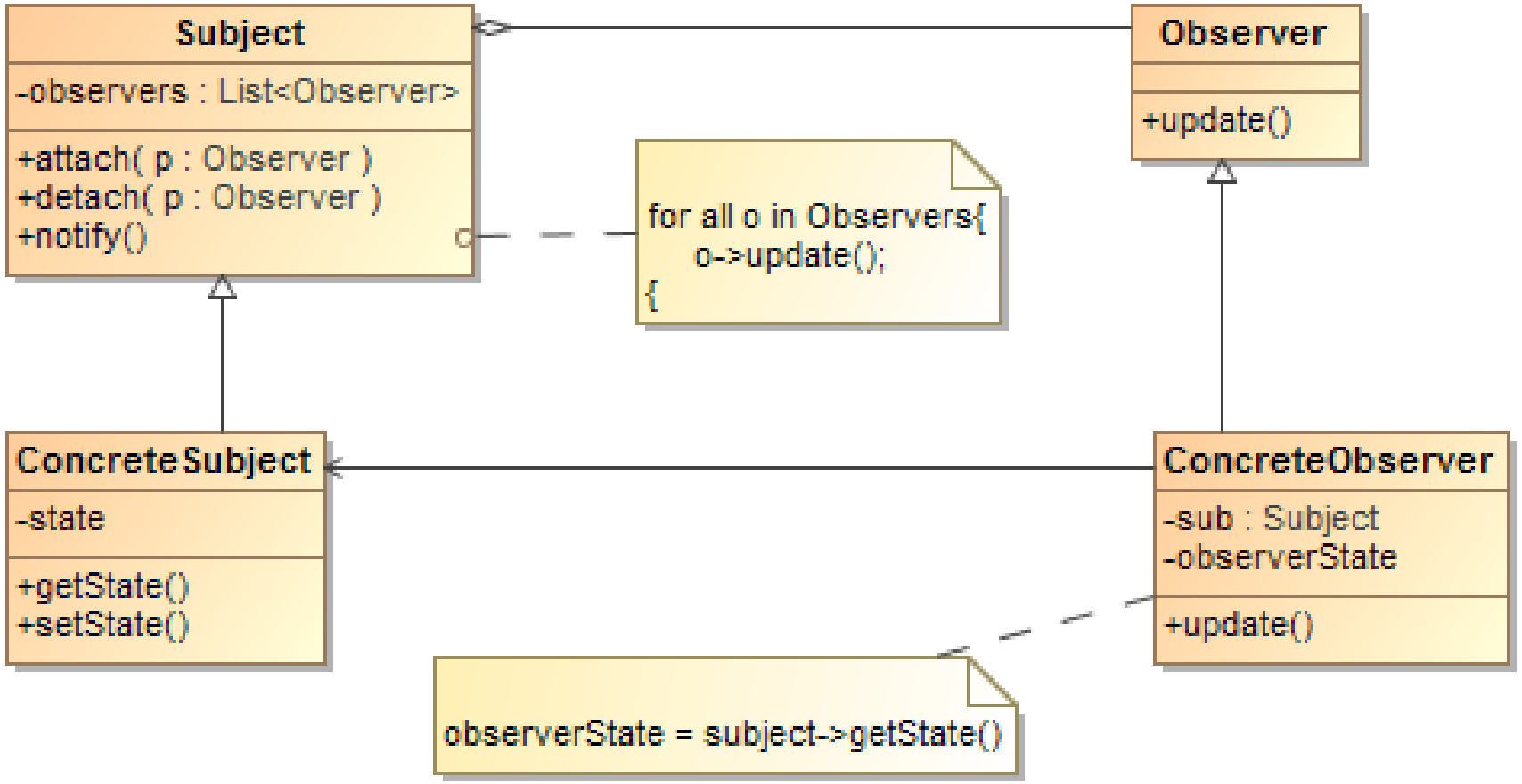
Variantai:

- **A)** Atributai tank, plane, ship
  B) Sąrašai List<Unit>
  C) Team Ieško visų Unit programoje
  D) Unit'ai patys nuolat klausia Team



Kaip GoF sprendžia informavimo apie pokyčius problemą

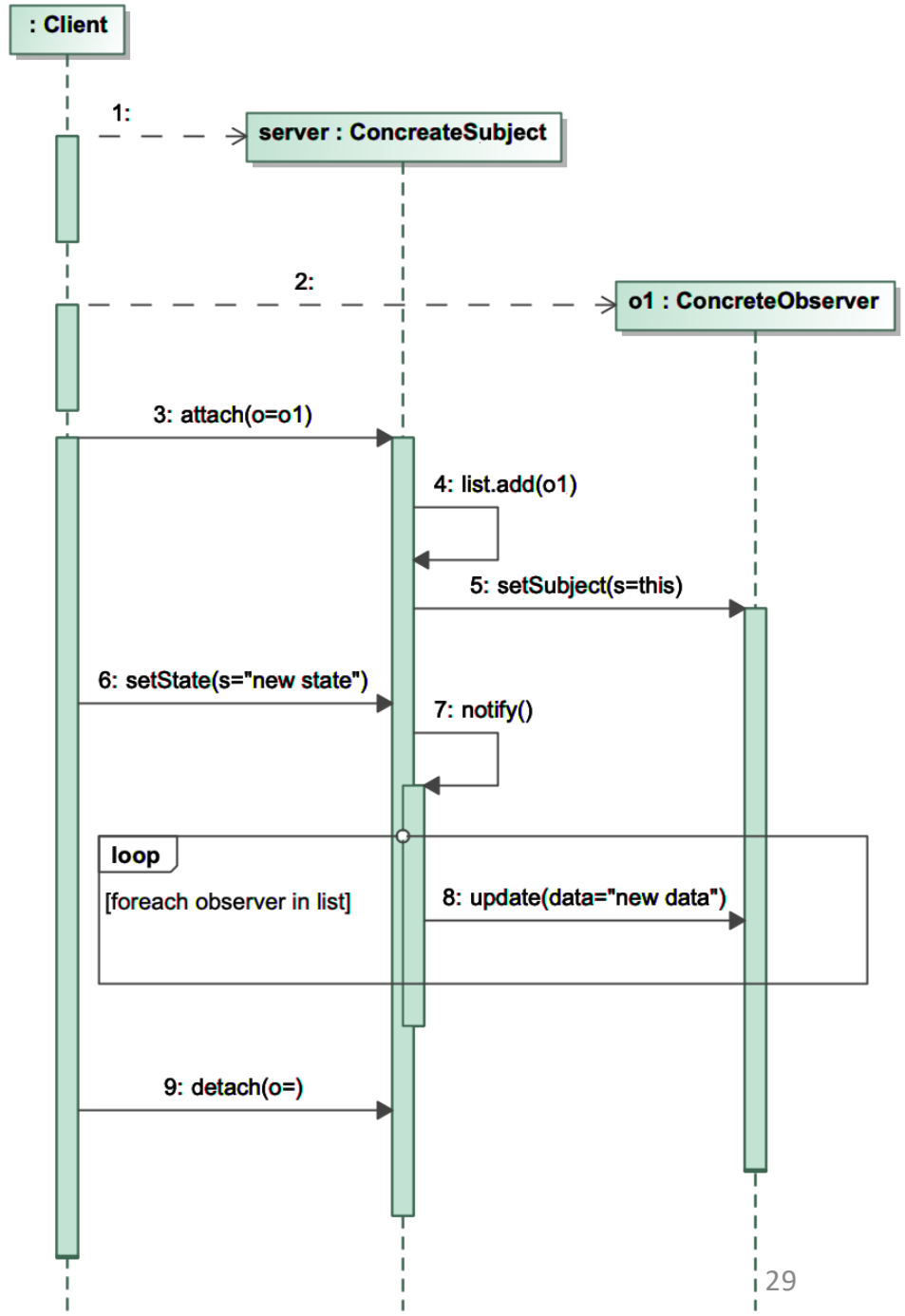
- Observer's structure (GoF)



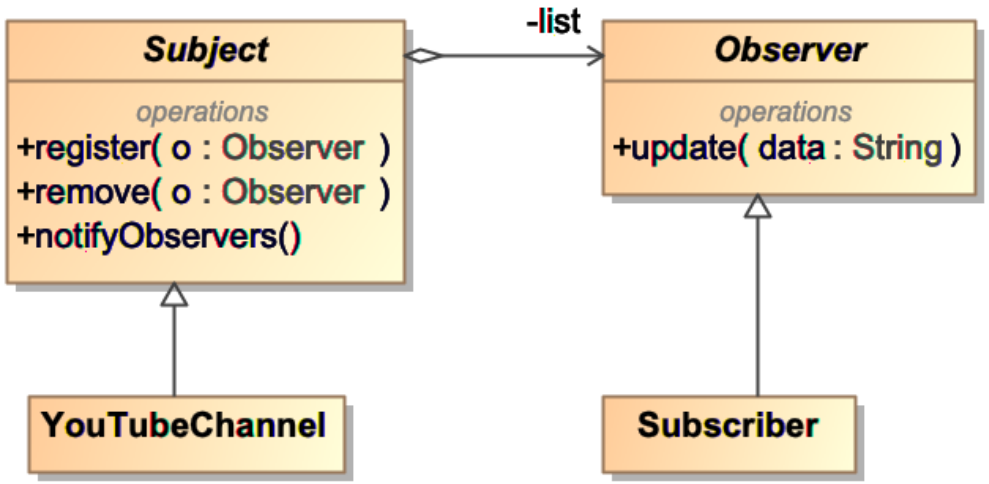
Per 10 sek. raskite trūkumus

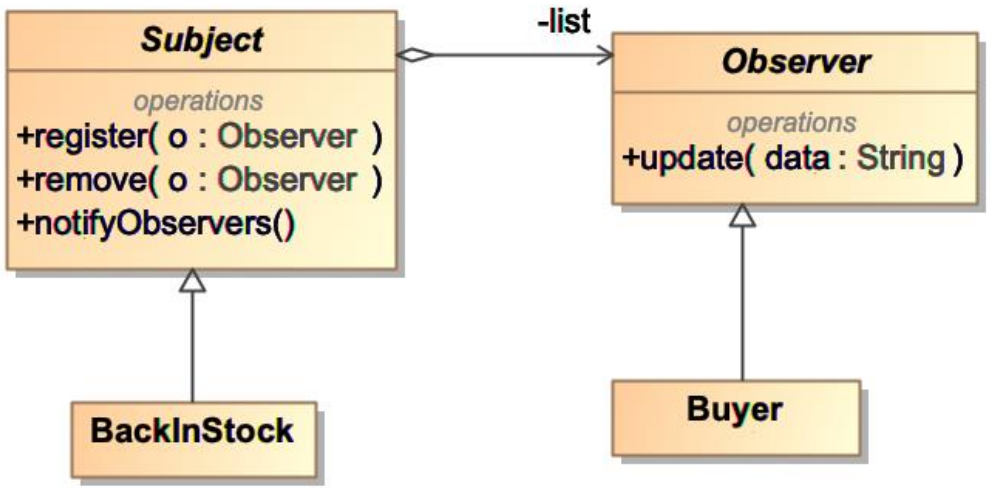
Kaip veikia Observer?

- Class diagram atsako: **Kas su kuo susiję?**
- Sequence diagram: **Kas vyksta laike?**



Kaip taikysime Observer šabloną Youtube ir Laukiamos prekės atvejais?









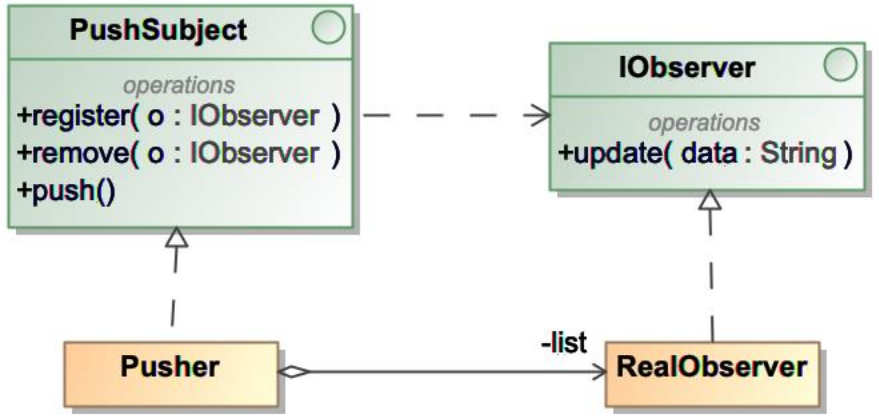
Taigi, kada mums praverčia Observer?

- kai **vieno objekto pokytis svarbus daugeliui kitų**;
- kai **nežinome, kiek objektų** norės gauti pranešimus;
- kai norime stebėtojus **pridėti ar pašalinti runtime**;
- kai Publisher neturėtų žinoti konkrečių Subscriber klasių.

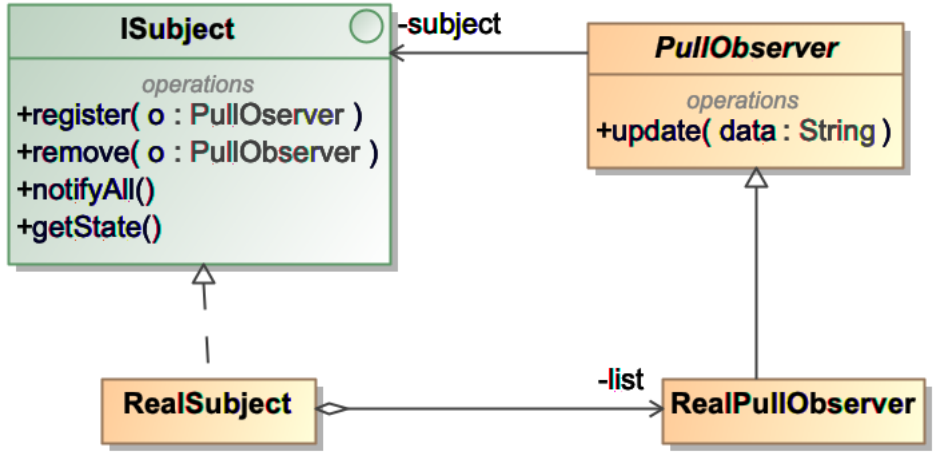
Mūsų žaidime: Team duoda komandą → visi užsiregistravę Unit'ai ją gauna.

Perdavimo modeliai (Push vs Pull)

Push modelis (Subjektas stumia duomenis)



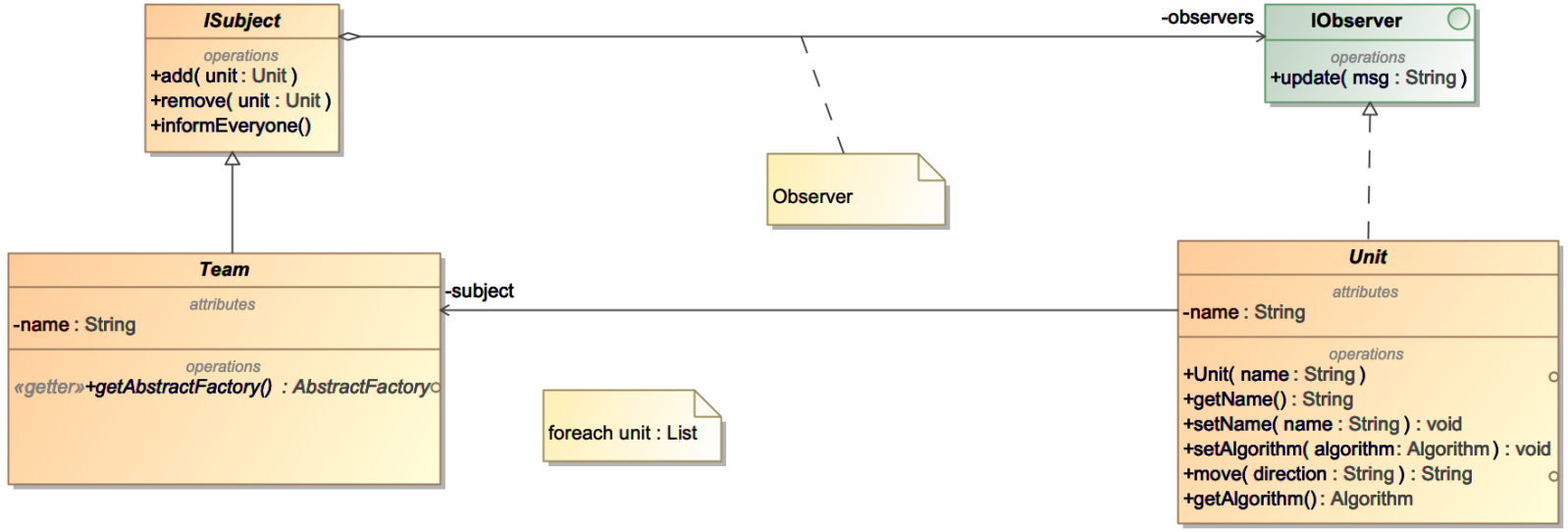
Pull modelis (Stebėtojas pats pasiima duomenis)



Ką rinktumėtės savo žaidimui?

•Kur per mūsų laboratorinius tinka taikyti Observer šabloną?

- Send/receive move commands from team to all its units.
- Team → Units



Ar Observer veikia tik vienos programos viduje?

Kliento – Serverio komunikacija:

- SignalR
- Websocket

- **GoF nėra 1994 metų muziejus.**



Observer Pros and Cons

✅ Loose coupling: the subject doesn't know anything about the Obervers

✅ Open/Closed Principle. You can introduce new subscriber classes without having to change the publisher's code (and vice versa if there's a publisher interface).

✅ You can establish relations between objects at runtime.

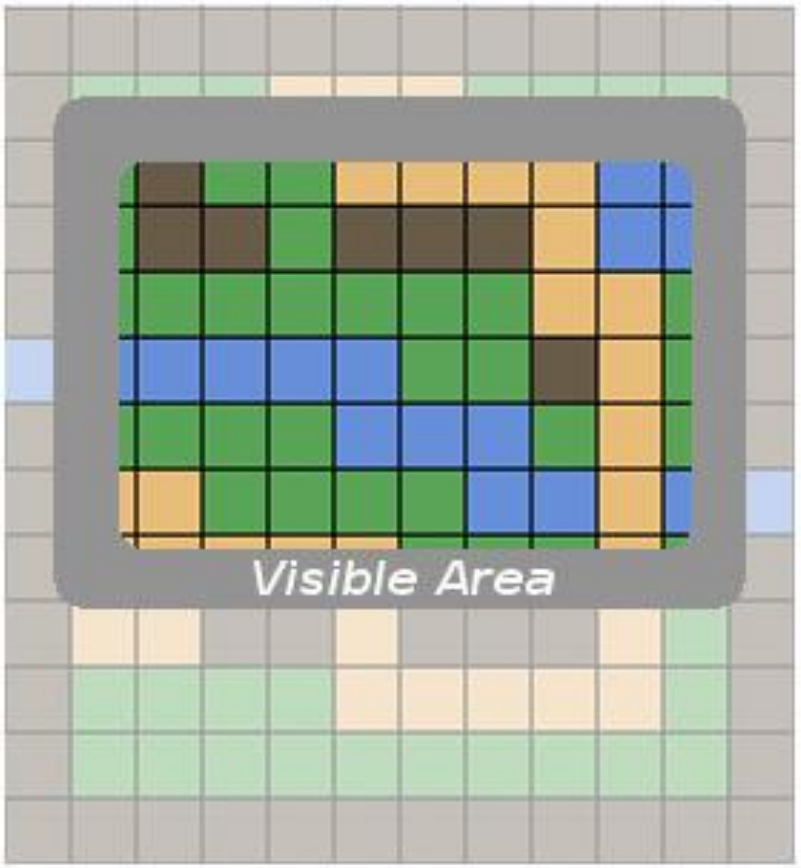
❌ The Subject (Publisher) may send updates that don't matter to the Observer (Subscriber).

❌ Subscribers are notified in random order.

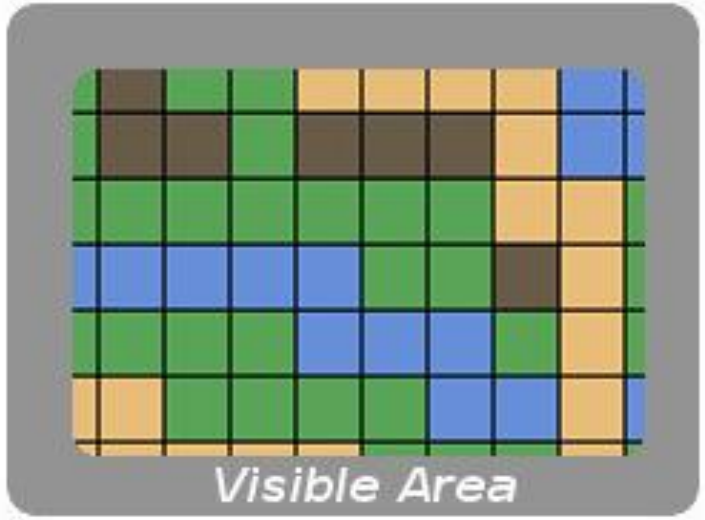
Ar žaidėjui reikia žinoti, kas vyksta kitoje žemėlapio pusėje?

Optimize:
`notifyAll()`

vs
`notifyRelevant()`



Without Culling
(All map tiles rendered)



With Culling
(Only visible tiles rendered)

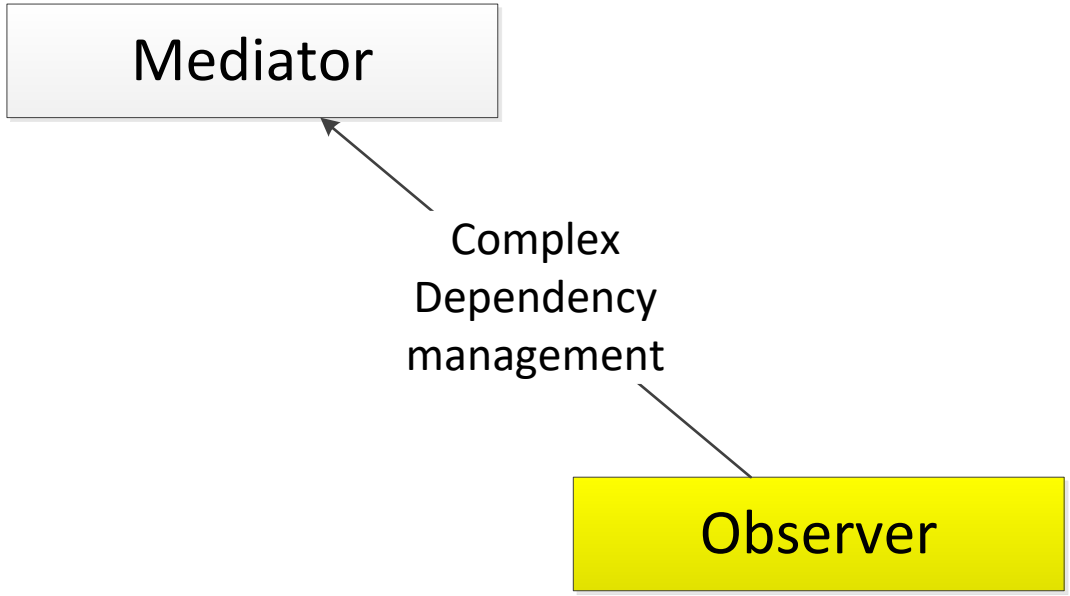
Observer išsprendė priklausomybes. Bet ar neišauginome pranešimų srauto?

- Observer mastelio problemos pavyzdys: Programinės įrangos atnaujinimai - **kas būtų, jei milijonai klientų tuo pat metu gautų pranešimą bei pradėtų siųstis update**
- **Kodėl Microsoft/Apple/kiti nepraneša milijonams kompiuterių tą pačią milisekundę?**

Observer: related patterns

- Mediator: By encapsulating complex update semantics, the ChangeManager acts as mediator between subjects and observers.

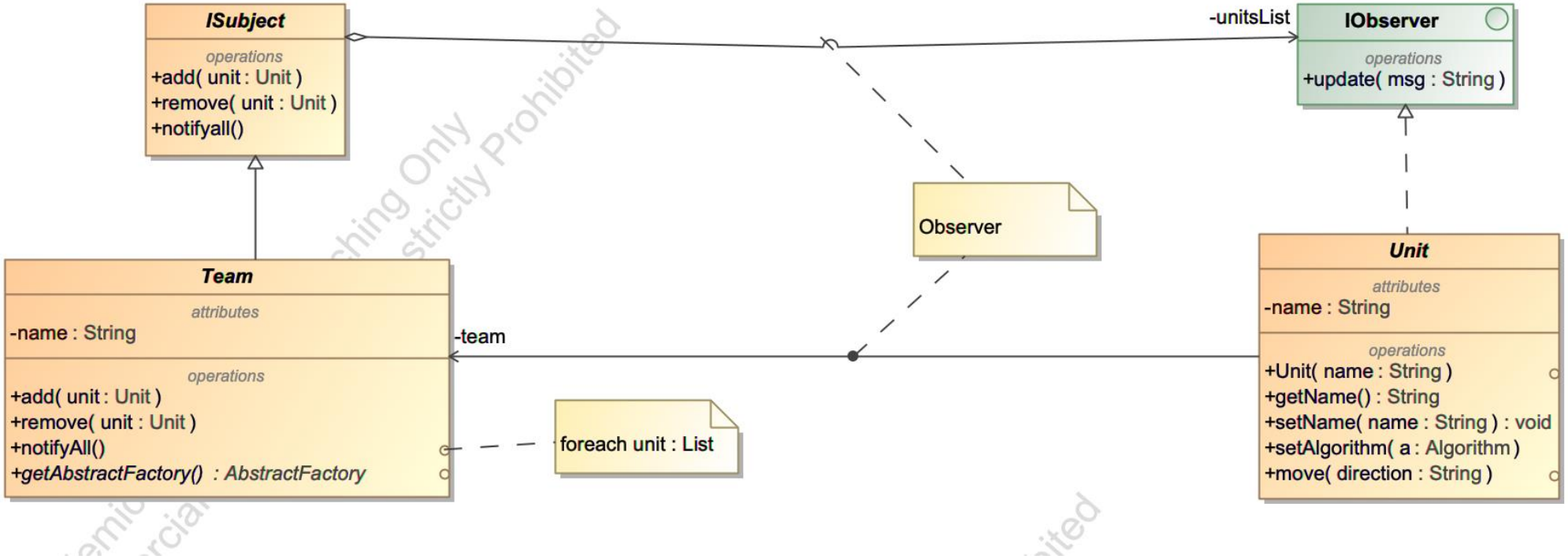
- Singleton: may use the Singleton pattern to make it unique and globally accessible.



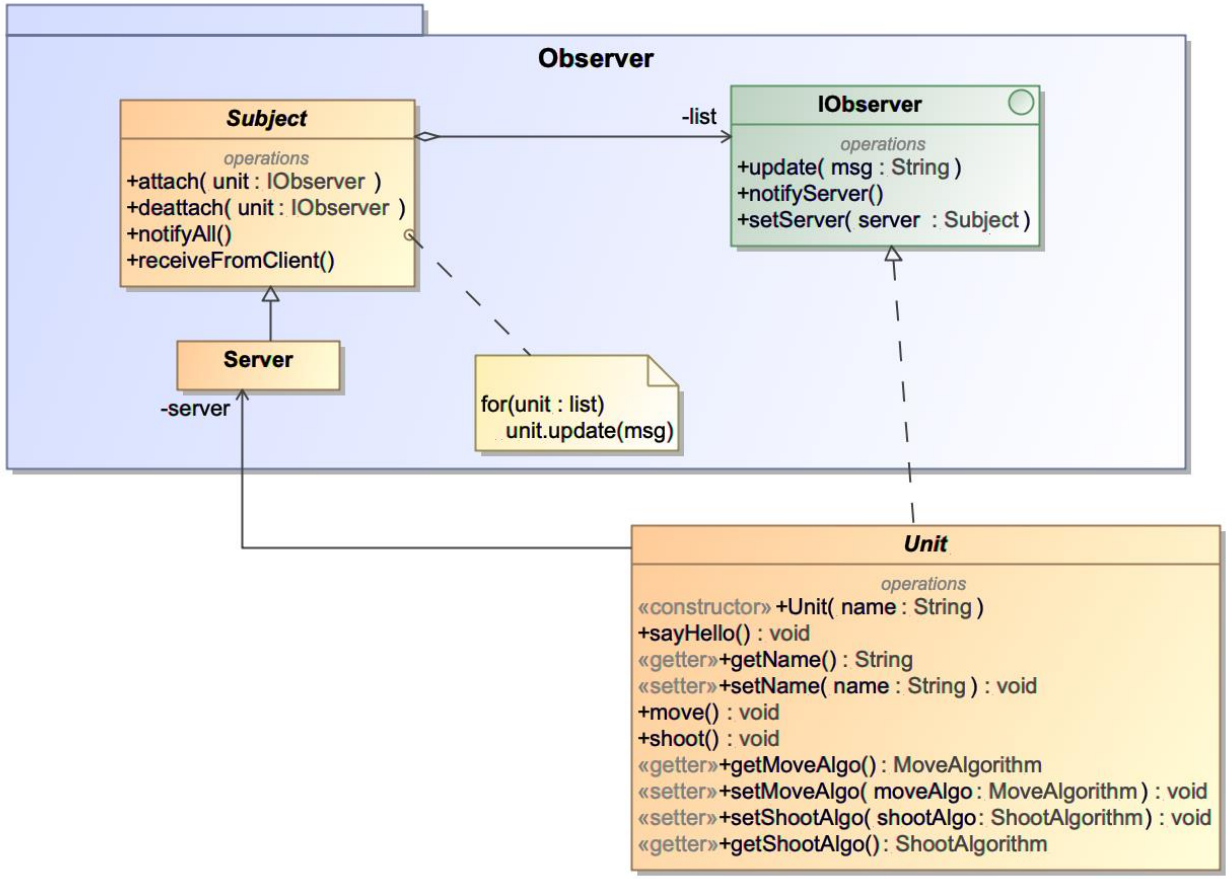
Observer: Example to code

- Send/receive move commands from team to all its units

Created during lecture: Observer



Observer example



Kuo šiandien iš esmės skyrėsi Strategy ir Observer?

- **Strategy:** Keičiu **Kaip vienas objektas elgiasi**.
  - Player → Run / Swim / Fly

- **Observer:** Sprendžiu **Kaip daug objektų sužino apie pokytį**.
  - Team says to units “move” → Tank / Plane / Ship

Klausimai?